

Lab 07 Template

Part 01

1. Print the `part01_ciphertext.bin` file and include a screenshot of the result in your lab report. Use `xxd -b <outputFile>`
(5 points total)

```
cpre3310@cpre3310:~/lab07/part01$ xxd -b part01_ciphertext.bin
00000000: 11011110 11011011 11000011 11010010 00110000 10011010  ....0.
00000006: 00011011 10100101 01111111 10111100 10011011 11110100  ....
0000000c: 10110010 10001011 10111110 00111101 00010101 11011010  ...=..
00000012: 01101101 11001011 10110110 01010110 00001111 00001000  m..V..
00000018: 00011100 11001100 00011001 10100000 01110100 00111011  ....t;
0000001e: 10101011 01110111 10111111 11011010 01010110 00001011  .w..V.
00000024: 00010100 00100100 10010010 10010101 00010110 10100000  .$. ...
0000002a: 00001001 11101010 01010001 11001001 10100001  ....Q..
cpre3310@cpre3310:~/lab07/part01$
```

Part 02

2. Include a screenshot of the result of decrypting `part02_ciphertext.bin`.
(5 points total)

```
cpre3310@cpre3310:~/lab07/part02$ python3 ../part01/part01_02_aes_student_skel.py decrypt part02_key.bin part02_nonce.bi
n part02_ciphertext.bin part02_plaintext.txt
File 'part02_ciphertext.bin' decrypted successfully to 'part02_plaintext.txt'.
Decrypted content (ASCII): I hope you have a nice and relaxing break!
```

```
GNU nano 7.2                                     part02_plaintext.txt
I hope you have a nice and relaxing break!
```

3. Functional code for both encrypt (part01) and decrypt (part02) submitted to Canvas as `<netid>_part01_02_aes.py`
(10 points total)
4. Why is giving everyone the same symmetric encryption key and none inherently wrong?
(10 points total)

Ans) It is wrong to do this as it removes the secrecy of the key because in symmetric encryption, the security of the system depends on the key being known to the communicating parties only, so if everyone has the same key, then any person can decrypt messages, modify plaintext, or basically impersonate another user.

Also, if we reuse the same nonce with AES-GCM basically weakens the security of the encryption since Nonces are meant to be unique for each encryption operation, and reusing them can basically allow attackers to analyze patterns or recover information about the plaintext. Thus, using the same symmetric key and nonce for multiple users can make the encryption insecure and defeat the purpose of protecting the confidentiality and integrity of the data.

5. **What difficult problem about using symmetric keys does this demonstrate? Think about the two parties (you and me) having to share the same key. What protocol have we started to discuss in lecture that can solve this problem?**

(10 points total, 5 points each question)

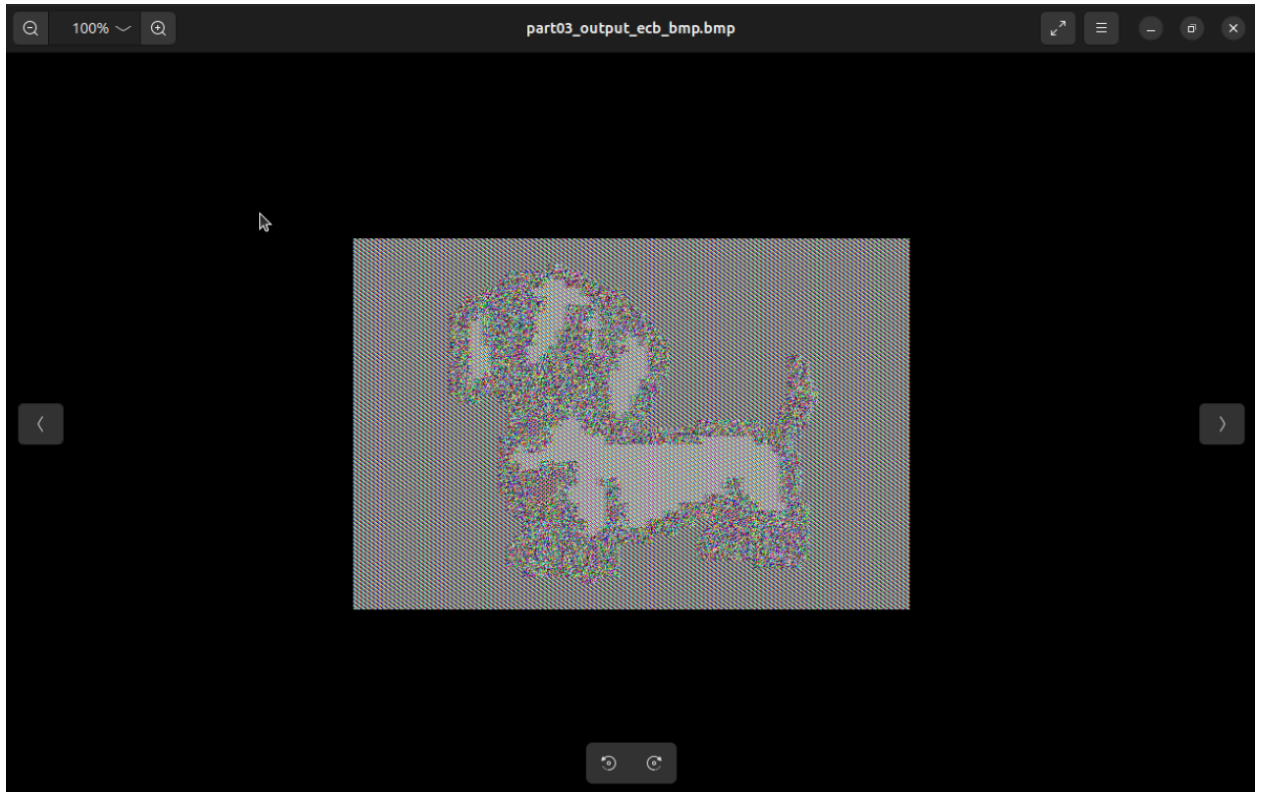
Ans) The problem that has been demonstrated over here is the key distribution problem. So, in symmetric encryption, both parties share the same key in order to encrypt or decrypt the plaintext messages, so over here the problem comes when the parties are sharing the keys with each other, as if the key gets transmitted on an insecure channel, then an attacker could intercept and gain access to all the encrypted communication.

The protocol that can basically solve this issue is the Diffie-Hellman exchange protocol, which basically allows 2 parties to securely generate a shared secret key over an insecure network without transmitting the key itself. Also, this secret can be used as a symmetric encryption for secure communication.

Part 03

6. **Open the two images and take a screenshot of them.**

(10 points)



7. **How does ECB mode handle these repetitive blocks, and why does this lead to visible patterns in the encrypted image?**

(10 points total, 5 points each question)

The ECB mode basically encrypts each block of plaintext independently by using the same key. If 2 plaintext blocks are identical then they will basically produce the same ciphertext block and images often contain repeating patterns, some identical blocks appear in multiple places and since ECB encrypts these blocks the same way each and every time, the structure of the original image still remains visible in the encrypted image, which reveals the patterns and the outlines.

8. **What effect does CBC mode have on repetitive structures in the image compared to ECB mode? How does the introduction of the Initialization Vector (IV) in CBC mode help in obscuring patterns that are visible in ECB mode?**

(10 points total, 5 points each question)

Ans) This mode basically hides repetitive patterns because each plaintext block is XORed with the previous ciphertext block before it gets encrypted and this means that even if 2 plaintexts are identical to each other, then also their ciphertext will be different if the previous ciphertext blocks are different. Over here, CBC also uses initialization vector (or IV) for the first block, which basically introduces randomness into the encryption process. The IV ensures that even identical images encrypted with the same key will basically produce different ciphertext outputs, which helps to obscure patterns and make the encrypted image look random

9. **Upload python code for part 03.**

(10 points)

Part 04

10. **In your own words, explain the CBC decryption occurring on the screen using the decryption notation from class.**

Please make a screenshot of your output and include it in the lab report. We need this since I used random values for the IV and key so each of your outputs will be different.

Ans) **Output**

```
cpre3310@cpre3310:~/lab07/part04$ python3 part04_padding_oracle_demo_v2.py
```

```
=== CBC Padding-Oracle Demo: Attack All Blocks (local only) ===
```

```
Ciphertext blocks (hex):
```

```
B0 (IV): 31271b753c8711ec9e7a5ee9300212b2  
B1: 80d9b529d07626942b4709f40517fe5d  
B2: fa36279f19d398a3b6b2910ece1b1c5f  
B3: 9ed5dc81a34ecf367319282fe879bbca  
B4: 7e08db46949edf6a010e8ccdf70a610f  
B5: ee28eb4cb854845a3ec1c37b733e3227
```

```
-----  
Attacking block index 1 (B1):
```

```
B0 (prev / IV if 0): 31271b753c8711ec9e7a5ee9300212b2  
B1 (target block) : 80d9b529d07626942b4709f40517fe5d  
D(B1) (derived): 754e7f5545e864cced1f3bc97b7b7edd  
Recovered (hex) : 44696420796f7520736565204b796c6f  
Recovered (ascii): Did you see Kylo  
Queries used for this block: 4096
```

```
-----  
Attacking block index 2 (B2):
```

```
B1 (prev / IV if 0): 80d9b529d07626942b4709f40517fe5d  
B2 (target block) : fa36279f19d398a3b6b2910ece1b1c5f  
D(B2) (derived): a08bd041f01752b4432e7ad477729d34  
Recovered (hex) : 2052656e206174206869732072656369  
Recovered (ascii): Ren at his reci  
Queries used for this block: 4096
```

```
-----  
Block 1:  
  Recovered (hex) : 44696420796f7520736565204b796c6f  
  Full slice (hex): 44696420796f7520736565204b796c6f  
  Recovered (ascii): Did you see Kylo  
  Full slice (ascii): Did you see Kylo
```

```
-----  
Block 2:  
  Recovered (hex) : 2052656e206174206869732072656369  
  Full slice (hex): 2052656e206174206869732072656369  
  Recovered (ascii): Ren at his reci  
  Full slice (ascii): Ren at his reci
```

```
-----  
Block 3:  
  Recovered (hex) : 74616c3f204920686561726420686520  
  Full slice (hex): 74616c3f204920686561726420686520  
  Recovered (ascii): tal? I heard he  
  Full slice (ascii): tal? I heard he
```

```
-----  
Block 4:  
  Recovered (hex) : 6b696c6c65642074686520536f6c6f2e  
  Full slice (hex): 6b696c6c65642074686520536f6c6f2e  
  Recovered (ascii): killed the Solo.  
  Full slice (ascii): killed the Solo.
```

```
-----  
Block 5:  
  Recovered (hex) : 101010101010101010101010101010  
  Full slice (hex):
```

```
-----  
Block 5:  
  Recovered (hex) : 101010101010101010101010101010  
  Full slice (hex):  
  Recovered (ascii):  
  Full slice (ascii):
```

```

Attacking block index 3 (B3):
  B2 (prev / IV if 0): fa36279f19d398a3b6b2910ece1b1c5f
  B3 (target block) : 9ed5dc81a34ecf367319282fe879bbca
  D(B3) (derived): 8e574ba0399ab8cbd3d3e36aee73797f
  Recovered (hex) : 74616c3f204920686561726420686520
  Recovered (ascii): tal? I heard he
  Queries used for this block: 4096

-----

Attacking block index 4 (B4):
  B3 (prev / IV if 0): 9ed5dc81a34ecf367319282fe879bbca
  B4 (target block) : 7e08db46949edf6a010e8ccdf70a610f
  D(B4) (derived): f5bcb0edc62aef421b7c087c8715d4e4
  Recovered (hex) : 6b696c6c65642074686520536f6c6f2e
  Recovered (ascii): killed the Solo.
  Queries used for this block: 4096

-----

Attacking block index 5 (B5):
  B4 (prev / IV if 0): 7e08db46949edf6a010e8ccdf70a610f
  B5 (target block) : ee28eb4cb854845a3ec1c37b733e322f
  D(B5) (derived): 6e18cb56848ecf7a111e9cdde71a711f
  Recovered (hex) : 10101010101010101010101010101010
  Recovered (ascii):
  Queries used for this block: 4352

-----

Finished attacking all blocks. Total oracle queries: 20736

=== Full plaintext (after decrypting whole ciphertext and unpadding) ===
Did you see Kylo Ren at his recital? I heard he killed the Solo.

```

The CBC encryption basically works by first decrypting a ciphertext block using the secret key and then it XORs the result with the previous ciphertext block, so for the first block, the Initialization Vector is used instead of a previous ciphertext block, so the formulas for CBC decryption are basically:

$$\begin{aligned}
 P_0 &= IV \oplus D(C_0, K) \\
 P_1 &= C_0 \oplus D(C_1, K) \\
 P_2 &= C_1 \oplus D(C_2, K)
 \end{aligned}$$

In the padding oracle attack, the attacker basically modifies the ciphertext blocks and observes whether the padding is valid or not, which basically allows the attacker to determine the intermediate decrypted values and recover the plaintext one byte at a time without knowing the secret key.

Then create a table where you complete the following.

(10 points)

Variable from slides	Corresponding variable from python output B0, B1, etc.	Hex value
IV	B0, since it represents the initialization vector and in CBC decryption, the IV is basically used with the decrypted block to recover the first plaintext block.	31271b753c8711ec9e7a5ee9300212b2
(C_0, K)	B1, since it is first ciphertext block which is decrypted using the secret key K during CBC decryption.	80d9b529d07626942b4709f40517fe5d
$D(C_0, K)$	The $D(B1)$ is the intermediate value obtained after decrypting ciphertext block B1 with the secret key before we do XOR.	754e7f5545e864cced1f3bc97b7b7edd
P	It is the recovered ascii block 1 after XORing the IV(B0) with the decrypted value $D(B1)$	Did you see Kylo

11. Why can we recover the plaintext with the CBC flaw?

(10 points)

Ans) The plaintext can be recovered since the system could reveal whether the padding of a decrypted message is valid or not, so when an attacker modifies the ciphertext blocks and submits them to the oracle, the system responds to them differently depending on whether is correct or not and by observing these responses, the attacker could determine the correct padding values and basically compute the intermediate decrypted values and using the

information, the attacker can recover the plaintext one byte at a time without knowing the encryption key.